Documento de Planejamento Lógico e Arquitetural — TaskTracker

1. Apresentação e Contextualização do Problema

Dados do Estudante

Item	Informação
Estudante	Klaus Müller Marques
Curso	Bacharelado em Ciência da Computação (8º Semestre)
Matrícula / RA	22301876
E-mail Institucional	klaus.muller.@sempreceub.com
Projeto	TaskTracker — Gestão Modular de Tarefas

Descrição do Problema

No cotidiano pessoal, acadêmico e profissional, a ausência de uma ferramenta estruturada para o gerenciamento de tarefas resulta frequentemente em perda de prazos, baixa produtividade e sobrecarga cognitiva. Sem um registro centralizado, priorização objetiva e acompanhamento do estado de execução, o indivíduo torna-se vulnerável ao esquecimento e à alocação ineficiente do tempo.

Proposta de Solução

O **TaskTracker** é um sistema de gerenciamento de tarefas projetado sob os princípios da arquitetura modular em camadas e boas práticas da Engenharia de Software. Ele permite a criação, listagem, consulta, edição, atualização de status, remoção e filtragem de atividades.
O objetivo da **Fase 1** do projeto é garantir:

- Clareza no planejamento lógico procedimental (fluxos e etapas de execução em alto nível).
- Separação rigorosa de responsabilidades (*Controller*, *Service*, *Repository*, *Model*).
- Fluxo de versionamento profissional utilizando Git e GitHub (*Feature Branching* e *Commits Semânticos*).

2. Decomposição do Sistema e Regras de Negócio

Entradas de Dados

1. **Identificador Único (id):** Número inteiro ($ID \in \mathbb{N}^*$) gerado automaticamente pelo sistema.
2. **Título (title):** Cadeia de caracteres (*String*) obrigatória com a síntese da tarefa.
3. **Descrição (description):** Texto descritivo opcional com o detalhamento das ações.
4. **Prioridade (priority):** Valor categórico restrito a um conjunto enumerado: "Alta", "Média" ou "Baixa".
5. **Data Limite (due_date):** Data válida no formato AAAA-MM-DD indicando o prazo limite.
6. **Status (status):** Estado de execução da tarefa: "Pendente", "Em Andamento" ou "Concluída".

Regras de Negócio e Validações (R_n)

- R_1 **— Obrigatoriedade do Título:** O campo title não pode ser nulo, vazio ou composto unicamente por espaços em branco.
- R_2 **— Restrição do Domínio de Prioridade:** Apenas os valores "Alta", "Média" ou "Baixa" são aceitos.
- R_3 **— Invalidação de Datas Retroativas:** A data limite due_date deve respeitar a condição $due_date \geq current_date$, onde $current_date$ Representa a data atual do cadastro ou edição.
- R_4 **— Integridade Referencial:** Ações de alteração, exclusão e busca requerem um id válido e existente no repositório.
- R_5 **— Transição de Status:** Os únicos status permitidos são "Pendente", "Em Andamento" e "Concluída".
- R_6 **— Persistência Segura:** Apenas dados devidamente validados pela camada de serviço (TaskService) chegam ao TaskRepository.

3. Fluxo de Execução dos Algoritmos (Alto Nível)

Abaixo estão descritas as etapas procedimentais e interativas de cada funcionalidade do **TaskTracker** em alto nível.

Módulo 1: Cadastro de Nova Tarefa (create_task)

- **Etapa 1:** O usuário seleciona a opção "Cadastrar Tarefa" no menu principal.
- **Etapa 2:** O sistema solicita o **Título** da tarefa. Caso o usuário envie em branco, o sistema exibe um alerta e solicita novamente (R_1).
- **Etapa 3:** O sistema solicita a **Descrição** (opcional). O usuário pode preencher ou pressionar Enter para ignorar.
- **Etapa 4:** O sistema solicita a **Prioridade** ("Alta", "Média" ou "Baixa"). O sistema valida a entrada e repete a solicitação caso a opção seja inválida (R_2).
- **Etapa 5:** O sistema pede a **Data Limite** (AAAA-MM-DD). É feita a verificação se a data é válida e se não é retroativa (R_3).
- **Etapa 6:** O sistema gera um ID único sequencial, define o status inicial como "Pendente" e envia os dados consolidados para a camada de serviço.
- **Etapa 7:** O repositório armazena a nova tarefa e exibe uma mensagem de confirmação de cadastro com o ID gerado.

Módulo 2: Listagem Geral de Tarefas (list_tasks)

- **Etapa 1:** O usuário seleciona a opção "Listar Tarefas" no menu principal.
- **Etapa 2:** O sistema consulta a camada de persistência para obter a lista completa de cadastros.
- **Etapa 3:** O sistema verifica se existem registros.
    - **Se a lista estiver vazia:** Exibe a mensagem informativa "[INFO] Nenhuma tarefa cadastrada no momento.".
    - **Se houver tarefas:** Percorre a lista e exibe cada registro formatado com ID, Título, Prioridade, Data Limite e Status.
- **Etapa 4:** O sistema finaliza a exibição e retorna ao menu principal.

Módulo 3: Consulta de Tarefa por ID (find_task_by_id)

- **Etapa 1:** O usuário escolhe a opção "Buscar Tarefa por ID".
- **Etapa 2:** O sistema solicita o código identificador (ID) desejado.
- **Etapa 3:** O serviço consulta o repositório procurando por uma tarefa com o ID fornecido (R_4).
- **Etapa 4:** O sistema valida o retorno:
    - **Se não encontrar:** Exibe um aviso de que a tarefa não existe no sistema.
    - **Se encontrar:** Apresenta na tela todos os detalhes da tarefa (ID, Título, Descrição

detalhada, Prioridade, Prazo e Status atual).

Módulo 4: Alteração de Status da Tarefa (update_task_status)

- **Etapa 1:** O usuário escolhe a opção "Atualizar Status".
- **Etapa 2:** O sistema solicita o ID da tarefa que terá seu progresso atualizado.
- **Etapa 3:** O sistema verifica se o ID existe. Se não existir, exibe mensagem de erro e cancela a operação (R_4).
- **Etapa 4:** Se a tarefa for localizada, exibe o status atual e apresenta as opções disponíveis: 1 - Pendente, 2 - Em Andamento, 3 - Concluída.
- **Etapa 5:** O usuário digita o número correspondente ao novo status desejado (R_5).
- **Etapa 6:** O serviço atualiza a propriedade do registro, o repositório salva a alteração e confirma a atualização para o usuário.

Módulo 5: Edição Completa de Tarefa (edit_task)

- **Etapa 1:** O usuário clica na opção "Editar Tarefa".
- **Etapa 2:** O sistema solicita o ID da tarefa a ser modificada e verifica se ela existe (R_4).
- **Etapa 3:** O sistema exibe o **Título** atual e pede o novo valor. Se o usuário apenas pressionar Enter, mantém-se o valor anterior.
- **Etapa 4:** O sistema pede a nova **Descrição**. Se deixado em branco, o texto original é preservado.
- **Etapa 5:** O sistema solicita a nova **Prioridade**. Se o usuário digitar um valor válido, ele é atualizado (R_2).
- **Etapa 6:** O sistema solicita a nova **Data Limite**. Se informada uma data futura válida, ela é atualizada (R_3).
- **Etapa 7:** As alterações são validadas e enviadas ao repositório, exibindo mensagem de sucesso.

Módulo 6: Exclusão / Remoção de Tarefa (delete_task)

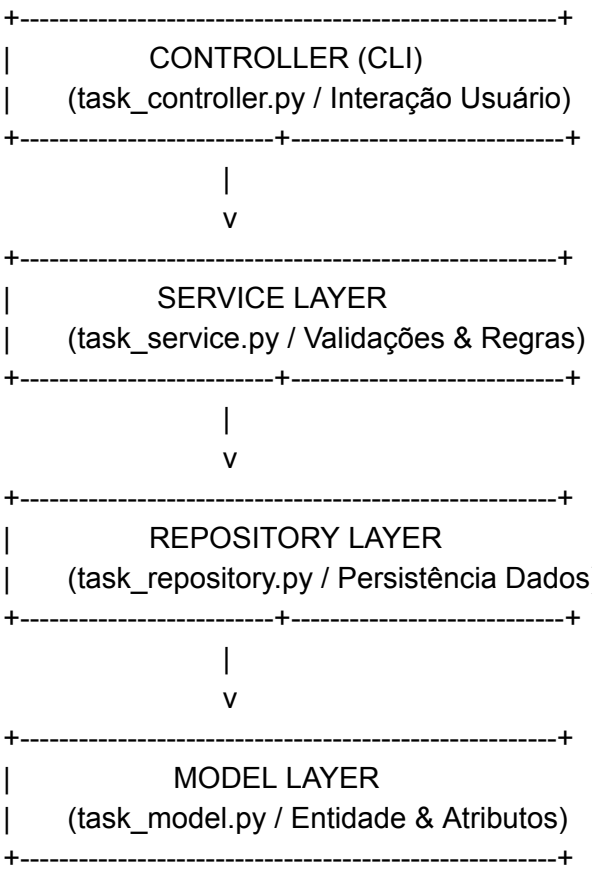
- **Etapa 1:** O usuário seleciona a opção "Remover Tarefa".
- **Etapa 2:** O sistema pede o ID da tarefa que deve ser excluída.
- **Etapa 3:** O sistema busca o registro e, se encontrado, exibe o título da tarefa pedindo uma confirmação do usuário (S/N).
- **Etapa 4:** O sistema avalia a confirmação:
    - **Se responder "S":** O repositório remove o registro definitivamente e exibe confirmação de exclusão.
    - **Se responder "N" ou inválido:** A operação é cancelada com segurança sem alterar os dados.

Módulo 7: Filtragem por Prioridade (filter_tasks_by_priority)

- **Etapa 1:** O usuário escolhe a opção "Filtrar Tarefas por Prioridade".
- **Etapa 2:** O sistema pergunta qual categoria deseja visualizar ("Alta", "Média" ou "Baixa").
- **Etapa 3:** O serviço solicita todos os registros ao repositório e seleciona apenas aqueles cujo campo priority coincida com a opção informada.
- **Etapa 4:** O sistema apresenta na tela o subconjunto de tarefas encontradas ou uma mensagem avisando que não há tarefas nessa prioridade.

4. Arquitetura de Software e Workflow Git

Arquitetura em Camadas (Layered Architecture)



- **controllers/task_controller.py:** Gerencia a interface por linha de comando (CLI), recebe inputs do usuário e exibe os retornos e menus.
- **services/task_service.py:** Centraliza a inteligência do negócio, validação das regras $R_1 \dots R_6$ e captura/tratamento de erros.
- **repositories/task_repository.py:** Encapsula o acesso aos dados e abstrai o armazenamento em memória/disco.
- **models/task_model.py:** Define a estrutura de dados e propriedades da classe Task.
- **main.py:** Ponto de entrada (*Bootstrapping*) responsável pela injeção de dependências e inicialização da aplicação.

Estrutura do Repositório GitHub

```
tasktracker-projeto/
├── README.md                 # Documentação explicativa do projeto
├── .gitignore                # Arquivo de exclusão de artefatos temporários
├── docs/
│   └── planejamento_logico.pdf # Documento oficial de planejamento da Fase 2
└── src/
    ├── main.py               # Script de inicialização do programa
    ├── controllers/
    │   └── task_controller.py  # Camada de Interface do Usuário (CLI)
    ├── services/
    │   └── task_service.py     # Camada de Regras de Negro e Validações
    ├── repositories/
    │   └── task_repository.py  # Camada de Persistência e Acesso a Dados
    └── models/
        └── task_model.py       # Modelo da Entidade Task
```

Estratégia de Versionamento Git

1. **Branch Principal (main):** Mantida estritamente com código testado, estável e revisado.
2. **Feature Branching:** Cada funcionalidade é desenvolvida em uma ramificação própria (ex.: feature/camadas-e-modelos, feature/validacao-datas).
3. **Commits Semânticos:** Padronização das mensagens de commit conforme o padrão internacional:
    - feat(...): Criação de nova funcionalidade.
    - fix(...): Correção de erros ou ajustes de bugs.
    - refactor(...): Reestruturação de código sem alteração do comportamento.
    - docs(...): Alterações e melhorias na documentação do projeto.

5. Entrega da Apresentação em Vídeo

- **Link da Apresentação no YouTube:**

Documento atualizado por Klaus Müller Marques para o Bootcamp de Desenvolvimento — Fase 2.